

DAT12-1

Introduction to Data with Spreadsheet

[Module page](#)

Version of 13 September 2026

About this textbook

This year, for the first time, every Bachelor course at Albert School has its own textbook, thanks to your teachers and AI.

- The textbook is a help for working on the course at home, after the class.
- The course as taught in class remains the reference, and prevails over the textbook wherever the two differ.
- This is a first version. Help us improve it by sharing your feedback with your teacher.

Contents

Preface	4
1. The Spreadsheet Environment	5
1.1. A grid that means something	5
1.2. Entering and editing data	5
1.3. What the cell really holds: type versus formatting	6
1.4. Laying the table out so everything else works	6
2. Core Functions	8
2.1. A formula is a question asked of the data	8
2.2. Copying a formula: relative and absolute references	8
2.3. Summarising a column: SUM and AVERAGE	9
2.4. Branching on a condition: IF	10
2.5. Counting what matches: COUNTIF	10
3. Looking Up Related Values	12
3.1. When the value you need lives in another table	12
3.2. VLOOKUP	12
3.3. INDEX and MATCH	13
4. Sorting and Filtering	15
4.1. Making a pattern visible	15
5. Data Quality at the Spreadsheet Level	17
5.1. The errors you can see if you look	17
6. Pivot Tables	19
6.1. Grouping, made interactive	19
7. Visualization	21
7.1. The right chart answers the question; the wrong one misleads	21
8. From Business Question to Spreadsheet Answer	24
8.1. The whole course, in three steps	24
9. Documentation and Reproducibility	26
9.1. An analysis no one can repeat is not finished	26

Preface

Welcome to the first course of your data education. It is, on purpose, the most humble one. Before you train a model, query a database, or write a line of Python, you will spend this whole course inside a tool you already half know: the spreadsheet. No prior experience is assumed. What **is** assumed is a willingness to slow down.

The one habit this course builds is short to say: **look at the data before you touch it**. A beginner reaches straight for a formula, a chart, an average. Experienced data workers first do something that looks like nothing — they scroll, they sort, they read a column, they ask “does this even make sense?” — before computing anything. They have learned that a number computed from dirty data is worse than no number, because it looks confident while being wrong; and that a chart answering the wrong question misleads exactly as much as a wrong calculation.

A spreadsheet is the right place to learn this, because everything is visible. Nothing is hidden in a pipeline. Every value sits in a cell you can click on, and every common mistake — a number stored as text, a repeated row, a stretched axis — is right there on the screen, once you have trained your eye to see it. That training is the real subject of the course. By the end you will be able to take a real business question, answer it using only spreadsheet operations, and write down your work so that a colleague can repeat it without asking you anything.

Remark (How to read this book). The step-by-step boxes use the names common to the three widely used spreadsheets — Microsoft Excel, Google Sheets, and LibreOffice Calc. Where a name differs between them, both are given. The example money amounts use a neutral currency unit written ₹ ; nothing in any analysis depends on which real currency it stands for. Dates are written in the international YYYY-MM-DD form (for example 2026-01-04) so they read the same everywhere.

1. The Spreadsheet Environment

1.1. A grid that means something

Open any spreadsheet and you see a grid: columns labelled with letters (A, B, C, ...) and rows numbered (1, 2, 3, ...). Each rectangle is a **cell**, named by its column then its row — B4 is the cell in column B, row 4. This naming is the whole reason spreadsheets work: every value has an address, so every formula can point to it.

Two parts of the interface do the naming and are worth finding right now, because the rest of the course uses them constantly:

Definition 1 (Name Box and formula bar). The **Name Box** sits at the top-left and shows the address of the cell you have selected (for example D2). The **formula bar**, just to its right, shows what that cell really **contains** — a typed value, or the **formula** behind a displayed result. The cell shows the answer; the formula bar shows how the answer was made.

Name Box formula bar — shows the stored formula

E2	=C2*D2				
----	--------	--	--	--	--

	A	B	C	D	E
1	Date	Product	Units	Price	Total
2	2026-01-01	Coffee	56	3.50	196.00
3	2026-01-01	Tea	25	2.80	70.00

selected cell E2: shows the value 196.00, while the formula bar shows the formula

Figure 1: The environment. The selected cell E2 displays its computed value (196.00), while the formula bar reveals that the value comes from the formula =C2*D2 (units times price). The Name Box confirms which cell is selected. Column letters and row numbers give every cell its address.

1.2. Entering and editing data

You put a value into a cell by clicking it and typing. Press Enter to confirm and move down; press Tab to confirm and move right; press Escape to cancel. To change a cell later, double-click it (or select it and press F2) to edit in place, or simply retype to replace it.

Method 1 (Enter and format a value correctly).

1. Click the target cell; type the value; press Enter.
2. Type a **number** as plain digits (1200), with no spaces and no currency symbol inside the cell.
3. Type a **date** in a form the spreadsheet recognises (2026-01-04); it should align to the right, showing it was understood as a real date.
4. To change only how a value **looks** — decimals, a currency symbol, a date style — select the cell and open the number-format control (in Excel and Calc: the **Home** / **Format** → **Cells** menu; in Google Sheets: **Format** → **Number**). Formatting changes the display, never the stored value (Figure 2).

1.3. What the cell really holds: type versus formatting

Click a cell and the spreadsheet must silently decide what kind of thing it holds — and that decision, not how the cell looks, governs whether later formulas work. Meet the trap first, then name it.

Example — The number that will not add up. You type 1 200 into one cell (with a space, the way you might write it by hand) and 1200 into another. On screen they look almost the same. But the space makes the first one **text**, while 1200 is a **number**. Ask for their sum and — tested in a real spreadsheet — you get 1200, not 2400: the text cell is silently treated as absent. Nothing turned red, nothing warned you. The total is simply wrong, and looks fine.

That invisible decision — the cell's **type** — is the single most common beginner bug in spreadsheet work, so we name it precisely.

Definition 2 (Type versus formatting). A cell's **type** is how the spreadsheet interprets its content: a **number**, a **date**, or **text**. **Formatting** controls only how that content is **displayed** (decimal places, a currency symbol, a date style); it never changes the stored value. Two cells can look identical on screen and still be of different types.

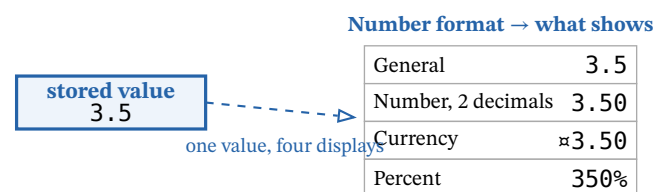


Figure 2: Formatting changes only what a cell **shows**, never what it **stores**. The single stored number 3.5 can display as 3.5, 3.50, ≈3.50, or 350% depending on its number format — and a later SUM still adds the same 3.5 in every case.

Pitfall. A number stored as text is skipped by SUM and AVERAGE, and a date stored as text will not sort by time — "10/01" sorts **before** "9/01" because text is compared character by character, not as dates. There is a reliable tell: numbers and dates align to the **right** of a cell by default, plain text to the **left**. A column of "numbers" that hugs the left edge is a column of text in disguise.

	A	B	C
1	Product	Units	Price
2	Coffee	12	3.50
3	Tea	7	2.80
4	Cocoa	9	4.00

left-aligned → stored as *text*, ignored by SUM

Figure 3: Numbers align right by default. The 9 in the Units column is left-aligned, betraying that it was stored as text; SUM and AVERAGE will quietly skip it.

1.4. Laying the table out so everything else works

Almost every operation in this course assumes one quiet convention.

Definition 3 (Tidy tabular layout). Data is laid out as a **table** in which each **column** is one variable (one kind of thing measured), each **row** is one record (one observation), and the first row holds **column headers** naming the variables. One cell holds one value.

Example — Reading a small sales table. A café records its sales. Here are the first rows exactly as the sheet holds them:

Date	Product	Units	Price (₺)	Total (₺)
2026-01-01	Coffee	56	3.50	196.00
2026-01-01	Tea	25	2.80	70.00
2026-01-01	Cocoa	21	4.00	84.00
2026-01-01	Juice	7	1.50	10.50

Each column is one variable (a date, a product, a count, a price, a total); each row is one sale. Because the layout is regular you can **read** it before computing: coffee sells more units than tea, and at a higher price. That reading — done by eye, before any formula — is the reflex this course is built around.

Exercises

1.1. Enter a small dataset of at least ten rows — say a few days of sales with date, product, units, and price. Make sure numbers are stored as numbers (right-aligned) and dates as dates. Then apply formatting so prices show two decimals and a currency symbol, without changing the stored values.

1.2. Deliberately type one number with a space inside it (like 1 200) so it is stored as text. Find it using the left/right alignment tell, then correct it so it becomes a real number. Confirm the fix by checking that the column now sums correctly.

1.3. Select any cell that holds a formula's result and read the formula bar. Write down, in your own words, the difference between what the cell **shows** and what the formula bar **shows**.

Answers. (1) Correctly entered numbers align right; formatting to two decimals and a currency symbol leaves the underlying number unchanged — a later SUM still works. (2) The text value aligns left and is skipped by SUM; retyping it without the space (or using the spreadsheet's "convert to number" helper) turns it into a real, right-aligned number and the column total rises by that value. (3) The cell shows the **value**; the formula bar shows the **formula or raw content** that produced it.

2. Core Functions

2.1. A formula is a question asked of the data

A **formula** begins with = and computes a result from values or, more usefully, from **cell references**. Typing =C2*D2 beside our sales table multiplies units (column C) by price (column D) to give a line total. Because it refers to cells rather than to the numbers themselves, correcting a value in C2 updates the total on its own — this is exactly the formula shown in the formula bar of Figure 1.

Method 2 (Type a formula).

1. Click the cell where the answer should appear.
2. Type =, then the expression — you may **click** a cell instead of typing its address, and the spreadsheet inserts it for you (=C2*D2).
3. Press Enter. The cell shows the result; the formula bar keeps the formula.

This chapter is not about memorising functions. It is about knowing **which** function a given question calls for. We meet five, each the right tool for a different shape of question.

2.2. Copying a formula: relative and absolute references

You rarely write a formula once. Having computed a total for one row, you want it for every row. You do not retype it — you **copy it down**.

Method 3 (Fill a formula down a column).

1. Select the cell holding the formula.
2. Grab the small square at its bottom-right corner — the **fill handle** — and drag it down over the rows you want; or copy the cell (Ctrl/Cmd + C) and paste into the range below.

	E
2	=C2*D2
3	=C3*D3
4	=C4*D4
5	=C5*D5

fill handle
drag down:
references shift

Figure 4: The fill handle — the small square at the selected cell's bottom-right corner. Dragging it down copies =C2*D2 into the rows beneath, and each copy's relative references shift with it: =C3*D3, =C4*D4, and so on.

When a formula is filled down, its cell references **move with it** (Figure 4). In row 2, =C2*D2; filled into row 3 it becomes =C3*D3, and so on. This automatic shifting is called a **relative reference**, and it is usually exactly what you want. But sometimes a formula must always point to **one fixed cell** — a single tax rate, a currency conversion, a target. For that you must stop the reference from moving.

Definition 4 (Relative and absolute references). A **relative reference** like B2 shifts when the formula is copied, so each copy points to its own row or column. An **absolute reference** freezes an address with the \$ sign: \$B\$2 never moves, B\$2 freezes only the row, \$B2 freezes only the column. Freeze exactly the part that must stay fixed as the formula is copied.

Example — One rate, applied to every row. A tax rate sits in cell G1. In a new column, F2, you write =E2*\$G\$1 to tax the first line total (the totals are in column E). Fill it down: E2 becomes E3, E4, ... (each row's own total), while \$G\$1 stays put, so every row is taxed by the one rate in G1. Had you written =E2*G1, filling down would slide the rate reference to G2, G3, ... — empty cells — and every total below the first would silently read zero. The \$ is what keeps the fixed cell fixed.

Pitfall. The commonest fill-down bug is forgetting the \$ on a value that should stay fixed. The first row looks right, so the error hides; the rows below quietly use the wrong (or empty) cell. After filling a formula down, click a cell far down the column and read its formula: check that the references that **should** have moved did, and the one that **should not** have moved did not.

2.3. Summarising a column: SUM and AVERAGE

Some questions are about a **whole column** at once — the café's 724 line totals in column E, taken together. Two of the commonest each have a one-word function.

Example — Total and typical revenue. With line totals in column E, =SUM(E2:E725) answers “how much did we take in altogether?” and =AVERAGE(E2:E725) answers “what was a typical sale worth?”. Over the café's six months of sales that is a total of $\approx 50,366.60$ across 724 rows, an average of ≈ 69.57 per row. The two questions differ, and so do the functions: reporting the sum when someone asked for the average — or the reverse — is a wrong answer even though every number in it is correct.

The two functions, in general:

Definition 5 (SUM and AVERAGE). SUM(range) adds the numbers in a range of cells. AVERAGE(range) returns their arithmetic mean — the sum divided by how many numbers there are. A **range** like E2:E100 names every cell from E2 down to E100.

Pitfall. AVERAGE ignores genuinely empty cells but treats a cell containing 0 as a real value. Given the numbers 10, 0, blank, 20, a real spreadsheet returns AVERAGE = 10 (the 0 is counted, the blank is not). Had that 0 truly meant “no data” and been left blank, the average would have been 15 instead. So **which** blanks were recorded as 0 is not a detail — it changes the answer. This is the first place the data-quality habits of Chapter 5 reach back and move a result.

2.4. Branching on a condition: IF

Sometimes a column should record a **decision** rather than a calculation — a label that depends on the value beside it.

Example — Flagging sales that need attention. A manager wants every sale whose revenue (column E) tops ≈ 120.00 marked for review. In a new column, `=IF(E2>120, "review", "")` writes “review” beside the big sales and leaves the rest blank. Filled down the whole sheet, it flags 112 of the 724 rows. The condition encodes the business rule; the two outcomes encode what to do about it. IF is how a human decision becomes a column.

The general form:

Definition 6 (IF). `IF(condition, value-if-true, value-if-false)` checks a condition and returns the first value when it holds, the second when it does not. For example `=IF(E2>100, "large", "small")` labels each sale.

2.5. Counting what matches: COUNTIF

A last question shifts from “how much” to “how **many**” — how many rows are coffee, how many sales beat a threshold.

Example — How many, not how much. `=COUNTIF(B2:B725, "Coffee")` counts how many rows are coffee sales — here 181 of them — while `=COUNTIF(E2:E725, ">120")` counts how many rows exceed ≈ 120.00 : 112. Note the difference from SUM: COUNTIF answers “how **many** rows?”, SUM answers “how much in **total**?”. Confusing a count with a sum is a classic error — “we had 112 big days” is a very different statement from “big days were worth 112.”

In general:

Definition 7 (COUNTIF). `COUNTIF(range, criterion)` counts how many cells in a range meet a criterion. The criterion can be a value (“Coffee”) or a comparison written in quotes (“>100”).

Pitfall (Function syntax depends on your region). This book separates a function’s arguments with commas: `SUM(E2:E725), IF(E2>100, "review", "")`. In many countries’ spreadsheet settings the separator is a **semicolon** instead — `IF(E2>100; "review"; "")` — and the decimal mark may be a comma. If a formula copied from this book gives an error, swap the commas between arguments for semicolons. The function names and logic are the same everywhere; only the punctuation follows your regional settings.

Exercises

2.1. Add a “Total” column to your dataset with `=units*price` in the first row, then fill it down. Click a cell near the bottom and confirm from the formula bar that both references moved with the row.

2.2. Put a single tax rate in one cell. In a new column, compute the taxed total for every row using an **absolute** reference to that rate cell. Then remove the \$ signs, fill down again, and describe exactly what goes wrong and why.

2.3. Using SUM, AVERAGE, IF, and COUNTIF, answer four questions about your dataset: a total, a typical value, a per-row flag, and a count of rows meeting a condition. For each, write one sentence explaining why that function — and not a similar one — is the right choice.

Answers. (1) Filled down, =C2*D2 becomes =C3*D3, =C4*D4, ...; each row multiplies its own units and price. (2) With =E2*\$G\$1 every row uses the one rate; drop the \$ and filling turns it into E3*G2, E4*G3, ... — the rate reference slides onto empty cells, so every row after the first reads zero. (3) SUM gives a total, AVERAGE a typical value, IF a per-row label, COUNTIF a count; a count and a total are different questions and swapping them gives a correct-looking but wrong answer.

3. Looking Up Related Values

3.1. When the value you need lives in another table

Often the value you need is in a **different** table. A sales sheet lists a product **code**; the price for each code sits in a separate price list. A **lookup** fetches the matching value across the two tables, so you never copy prices by hand.

Our café keeps this price list:

Code	Product	Price (¤)
C01	Coffee	3.50
T01	Tea	2.80
K01	Cocoa	4.00
J01	Juice	1.50

3.2. VLOOKUP

Definition 8 (VLOOKUP). `VLOOKUP(key, table, column, FALSE)` searches for **key** in the **first** column of **table** and returns the value from the numbered **column** of the matching row. The final **FALSE** demands an **exact** match — nearly always what you want.

Method 4 (Look up a price with VLOOKUP).

1. In the sales sheet, beside a row whose code is in A2, type `=VLOOKUP(A2, pricelist, 3, FALSE)`, where `pricelist` is the whole three-column price-list range.
2. A2 is the code to find; `pricelist` is the table to search; 3 is the column of the value you want (the price is the price list's third column); **FALSE** forces an exact match.
3. Press Enter, then fill the formula down so every sales row gets its price.

Price list

1: code 2: product 3: price

C01	Coffee	3.50
T01	Tea	2.80
K01	Cocoa	4.00
J01	Juice	1.50

A sale has code T01.

Find its price:

`VLOOKUP("T01", pricelist, 3, FALSE)`

returns **2.80**.

Figure 5: A lookup matches the key T01 in the first column of the price list and returns the value from the chosen column (here column 3, the price) of that row. `VLOOKUP` requires the key in the first column and can only read columns to its **right**.

Pitfall. `VLOOKUP` fails silently in two classic ways. First, the final **FALSE** is not optional decoration. With it, a lookup for a code that is not in the list returns `#N/A`, honestly reporting “not found”. Omit it and `VLOOKUP` quietly switches to **approximate** matching, a mode that only works when

the first column is sorted in ascending order — and our price list, like most, is not. On an unsorted column the result is **undefined**: approximate matching returns whatever its internal search happens to land on, and where it lands depends on the spreadsheet you use, so different programs — even different versions of one — can disagree. It can misprice even a code that **is** present. In the spreadsheet this book was checked with, `=VLOOKUP("K01", pricelist, 3)` without the `FALSE` — a lookup of Cocoa (¥4.00) — comes back as ¥3.50, Coffee's price; another program might return a different wrong price, or an error. That unpredictability **is** the danger. Always end `VLOOKUP` with `FALSE`. Second, `VLOOKUP` can only read columns to the **right** of the key; if the value you need sits to its left, `VLOOKUP` cannot reach it.

3.3. INDEX and MATCH

`VLOOKUP` is quick but boxed in: the key must be the first column, and the answer must sit to its right. A two-function pair removes both limits. It looks harder at first, so we build it up one piece at a time.

Definition 9 (INDEX and MATCH). `MATCH(key, range, 0)` returns the **position** of key within a single row or column — 1 for the first cell, 2 for the second, and so on (the 0 demands an exact match). `INDEX(range, n)` returns the **nth** value of a range. Feed one into the other and you have a lookup: `INDEX` fetches the value at the position `MATCH` found.

Example — Building an INDEX/MATCH lookup step by step. We want the price for code T01 from the price list. Unlike `VLOOKUP`, this pair points at **single columns**: codes is the code column and prices the price column (not the whole `pricelist` table).

1. `MATCH("T01", codes, 0)` looks down the code column and finds T01 in position 2.
2. `INDEX(prices, 2)` returns the **second** price, ¥2.80.
3. Nest them so the position feeds the fetch: `=INDEX(prices, MATCH("T01", codes, 0))`. Tested in a real spreadsheet this returns 2.8 — the same answer as `VLOOKUP`, but the key column may sit anywhere.

The pair's real advantage is reading **leftwards**. If the product **name** sits in a column to the **left** of the code, `=INDEX(names, MATCH("T01", codes, 0))` returns "Tea" — something `VLOOKUP` simply cannot do.

Exercises

3.1. Give each product in your dataset a short code and put codes and prices in a separate two-column price list. In the sales sheet, use `VLOOKUP(..., FALSE)` to fetch each row's price from its code, then fill it down.

3.2. Look up a code that does **not** appear in the price list, once with `FALSE` and once without it. Record both results and explain which is safer for real work and why.

3.3. Rebuild the same price lookup with `INDEX/MATCH`. Then rearrange the price list so the value you want sits to the **left** of the code, and show that `INDEX/MATCH` still works where `VLOOKUP` cannot.

Answers. (1) `=VLOOKUP(A2, pricelist, 2, FALSE)` (with the price in the two-column price list's second column) returns each row's price; filling down prices the whole sheet. (2) With `FALSE`, a missing code returns `#N/A` — an honest "not found". Without it, `VLOOKUP` switches to approximate matching, which is only defined on a first column sorted in ascending order; on an ordinary unsorted price list its result is unreliable — a wrong price, or an error, with no warning either way — so `FALSE` is the safe choice. (3) `=INDEX(prices, MATCH(A2, codes, 0))`, pointing `MATCH` at the

code column codes and INDEX at the price column prices, returns the same prices; because MATCH locates the row and INDEX fetches from **any** column, it also reads a value sitting left of the key.

4. Sorting and Filtering

4.1. Making a pattern visible

A thousand rows hide their patterns from the eye. Two operations — sorting and filtering — rearrange or hide rows until the pattern, or the anomaly, stands out. Neither computes anything new; they only change **what you see**, which is exactly the point of looking before touching.

Definition 10 (Sorting and filtering). **Sorting** reorders the rows by the values in one or more columns (smallest to largest, A to Z, oldest to newest, or the reverse). **Filtering** hides the rows that fail a stated condition, leaving the rest in view; the hidden rows are not deleted, only concealed, and clearing the filter brings them back.

Method 5 (Sort and filter a table).

1. Click any cell inside the table.
2. **Sort:** open **Data** → **Sort** (Excel, Calc) or **Data** → **Sort range** (Sheets), choose the column and the direction. Always let it sort the **whole table**, so each row stays together.
3. **Filter:** turn on the filter control — **Data** → **Filter** (Excel, Calc) or **Data** → **Create a filter** (Sheets). A small arrow appears on each header; click it to keep only the values or the range you want. Click it again to clear.

Example — The same table, two questions. Take the café’s six months of sales.

- **Sort** by total revenue, largest first: the biggest sales jump to the top, and an impossible value — a typo like 99999 — would surface at the very top where you cannot miss it.
- **Filter** to Product = Tea: only tea sales remain, isolating “how is tea doing?” from the noise of everything else.

Each operation answered a different question without changing a single value.

Pitfall. When you sort, sort the **whole table**, never a single column on its own. Sorting one column while its neighbours stay put scrambles every row — prices end up beside the wrong products — and, like the text-number trap, it leaves a sheet that looks tidy while being silently corrupted. If your spreadsheet warns that it found data “next to” your selection and offers to **expand** the selection, accept. If in doubt, check that one familiar row still holds together after the sort.

Remark (What about grouping?). A third question — “what is the **total** for each product, or each month?” — asks you to **group** rows that share a value and summarise each group. A plain spreadsheet has no one-click “group and total” button beside Sort and Filter. For a single group you can **filter** to it and read a SUM or AVERAGE of what is left; for grouping by every value of one or two columns at once, the right tool is the **pivot table**, which the next chapter is entirely about.

Exercises

4.1. Sort your dataset by a numeric column, largest first, and describe the single most interesting row at the top. Then deliberately insert one impossible value and confirm that sorting brings it to where you cannot miss it.

4.2. Turn on the filter control and filter to one category. State the question your filtered view answers, then clear the filter and confirm every hidden row returns.

4.3. Sort a single column **without** expanding to the whole table (make a copy of your data first). Describe precisely how the rows are now corrupted, and how the alignment or a familiar row reveals it.

Answers. (1) The largest-first sort puts the extreme rows on top; an inserted 99999 rises to the very top, which is exactly why sorting is a fast anomaly check. (2) A filter to one product answers “how is this product doing?”; clearing it restores all rows because filtering only hides, never deletes. (3) Sorting one column alone leaves the other columns in their old order, so each row now pairs the wrong values — a familiar record no longer holds together, which is the tell.

5. Data Quality at the Spreadsheet Level

5.1. The errors you can see if you look

Here the course's tagline pays off. Most data problems that wreck an analysis are visible at the spreadsheet level — **if** you inspect the data before computing. Three kinds account for the great majority.

Definition 11 (Three common data-quality issues).

- **Duplicates:** rows that repeat the same record, inflating every count and sum.
- **Inconsistent formats:** the same value entered as genuinely different text — a typo (Cofee for Coffee) or a stray trailing space (Coffee vs Coffee) — so that equal things are not recognised as equal. (Pure letter case is the exception: Cof fee and cof fee **are** treated as equal, as the pitfall below shows.)
- **Missing values:** cells left blank where a value was expected, which downstream functions may treat as zero, skip, or reject.

Example — One dirty extract, three problems. A 12-row extract of the café's February sales (Figure 6) hides all three. The Product column should hold just the four drink names, yet it holds 4 spellings of coffee alone — "Coffee", "coffee", "Cofee", "Coffee " — so the till's coffee sales are split 4 ways. A naive =COUNTIF(B2:B13, "Coffee") counts 2 of the 4 rows that are really coffee: it catches Coffee **and** coffee — COUNTIF ignores letter case — but misses the typo Cofee and the trailing-space "Coffee ", which are genuinely different text. Case is not what splits the count; the typo and the stray space are. Meanwhile 2 rows are exact duplicates of each other, and one Units cell was left blank.

	A	B	C	D	
1	Date	Product	Units	Price (€)	
2	2026-02-01	Coffee	12	3.50	
3	2026-02-01	coffee	9	3.50	← lower-case
4	2026-02-02	Cofee	7	3.50	← typo
5	2026-02-02	Coffee	5	3.50	← trailing space
6	2026-02-03	Tea	8	2.80	
7	2026-02-03	Tea	6	2.80	
8	2026-02-04	Cocoa	11	4.00	
9	2026-02-04	Juice		1.50	← blank
10	2026-02-05	Juice	7	1.50	
11	2026-02-05	Cocoa	9	4.00	
12	2026-02-06	Tea	10	2.80	] exact duplicate
13	2026-02-06	Tea	10	2.80	

Figure 6: The dirty February extract, with every defect marked. Four Product cells that all mean coffee are spelled four ways — one only in letter case (which COUNTIF forgives), one a typo, one with a trailing space (which it does not). One Units cell is blank, and rows 12–13 are an exact duplicate pair. Every flaw here is visible on the screen once you look.

Pitfall (Letter case is not an inconsistency COUNTIF sees). A tempting reflex is to treat Coffee vs coffee as a defect that hides rows from COUNTIF. It does not: COUNTIF and pivot grouping compare text **case-insensitively**, so Coffee and coffee are already counted and grouped as one. What they cannot forgive is text that genuinely differs — a typo (Cofee) or a stray space: Coffee is not Coffee, and =COUNTIF(range, "Coffee") skips it. Tidy the case for presentation if you like, but know it moves no count; fixing Cofee and stray spaces is what changes the numbers.

Method 6 (Find and correct the three issues).

1. **Duplicates:** sort the table so identical rows sit together, or use **Data → Remove duplicates** (Excel, Sheets) / **Data → More filters → Standard filter, “no duplications”** (Calc). Confirm the row count drops by the number you expected.
2. **Inconsistent formats:** sort the column so variants cluster, decide on one spelling, and replace the others (**Find & Replace** handles bulk fixes). Now a COUNTIF counts them all.
3. **Missing values:** filter the column to blanks. For each, **decide:** is the value genuinely unknown (leave it blank and exclude that row from the relevant total), or clearly recoverable from another column (fill it)? Record the decision.

Pitfall. Never silently delete a row or overwrite a value while cleaning. Each correction changes the result and must be **recorded** (Chapter 9), because someone repeating your work needs to know not just the final numbers but every judgement that produced them. Cleaning without a record is how an analysis becomes impossible to repeat — and impossible to defend.

Remark. Notice that cleaning needs no new tools. **Sorting** brings duplicates and variants together, **filtering** isolates blanks, and COUNTIF measures how widespread a problem is before you start fixing it. Data quality is not a separate skill; it is the earlier operations turned on the data itself.

Exercises

- 5.1. Take a deliberately dirty dataset (some duplicate rows, one category spelled several ways, a few blanks). Count the rows, remove the exact duplicates, and confirm the count dropped by the right amount.
- 5.2. Use COUNTIF to count one category **before** standardising its spellings, then standardise the variants and count again. Report both numbers and explain the gap.
- 5.3. For each blank in a column, decide whether to exclude or fill it, and write one line justifying each decision. Explain how an AVERAGE over the column would differ if the blanks had instead been recorded as 0.

Answers. (1) Removing the exact duplicates drops the row count by the number of extra copies (in the extract of this chapter, from 12 to 11). (2) Before standardising, COUNTIF(., "Coffee") already sees 2 — it counts Coffee and coffee alike, since case is ignored; merging the genuinely different Cofee and "Coffee " into Coffee raises it to 4. The gap is the typo and the stray space, not the case difference. (3) Blanks are skipped by AVERAGE; had they been 0, they would have been counted and dragged the average down, so the two choices give different results (as in Chapter 2).

6. Pivot Tables

6.1. Grouping, made interactive

Summarising one group by hand is fine once. But business questions multiply — revenue by month, then by month **and** product, then the average rather than the total — and redoing the summary each time is slow and error-prone. A **pivot table** is a machine for grouping that you steer by dragging fields.

Definition 12 (Pivot table). A **pivot table** summarises a source table by grouping rows according to one or more fields and computing an **aggregate** (count, sum, average, ...) of another field for each group. The chosen **row field** and **column field** become the edges of the result; each **body cell** holds the aggregate for that combination.

Method 7 (Build a pivot table).

1. Click any cell in the source table. Open **Insert** → **PivotTable** (Excel), **Insert** → **Pivot table** (Sheets), or **Insert** → **Pivot Table** (Calc).
2. A layout panel appears (Figure 7) with four areas — **Rows**, **Columns**, **Values**, and **Filters** — and a list of your field names.
3. **Drag** a field into **Rows** (here Product), another into **Columns** (here the month), and the field to summarise into **Values** (here Revenue).
4. In the **Values** area, set the aggregate to **Sum** (click the field to change it to Average, Count, and so on).

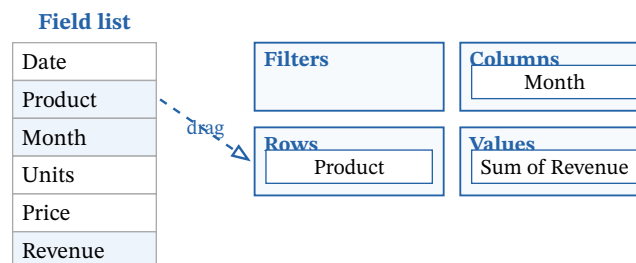


Figure 7: The pivot layout panel: a list of the source table’s fields on the left, and four drop areas on the right. You **drag** a field into an area — Product into **Rows**, Month into **Columns**, Revenue into **Values** (set to **Sum**) — and the pivot in Figure 8 is the result.

column field: month

row field: product	Σ Revenue (Σ)	Jan	Feb	Mar	Apr	May	Jun
	Coffee	6,062	4,984	4,928	3,899	3,273	2,363
	Tea	2,699	2,246	2,022	1,669	1,509	1,019
	Cocoa	2,552	1,996	1,944	1,552	1,308	1,004
	Juice	305	362	527	617	711	819

Figure 8: A pivot table with product as the row field, month as the column field, and summed revenue in the body. Each body cell answers “how much revenue, for this product, in this month?” — one grouping, read many ways. The highlighted cell is January coffee.

Example — From pivot cell to business sentence. The pivot in Figure 8 shows Coffee in January at ¥6,062.00. Building the table is mechanical; the value is in the **reading**. “Coffee earned ¥6,062.00 in January, more than twice tea’s ¥2,699.20 (2.25 times as much) — coffee carries our winter revenue” is a sentence a manager can act on. Reading **across** a coffee’s row shows its months falling from ¥6,062.00 toward summer, the seasonal decline of a hot drink. A pivot table that is built but not **interpreted** is just a smaller spreadsheet.

Pitfall. A pivot table inherits every flaw of its source. If Coffee and coffee (with a trailing space) both appear, the pivot lists them as **two** products and splits the revenue between them — a wrong answer produced by a correct machine. Clean the data (Chapter 5) **before** you trust a pivot.

Exercises

- 6.1.** Build a pivot table from your data with one row field, one column field, and summed revenue. Write one business sentence interpreting the single most striking cell.
- 6.2.** Change the aggregate from sum to average and explain how the business reading of the same table changes — what question does each version answer?
- 6.3.** Introduce one inconsistently-spelled category value into your source (for example a trailing space), rebuild the pivot, and describe exactly how the result becomes wrong and how you would spot it.

Answers. (1) Each body cell is the total for one row-by-column combination; the most striking cell is usually the largest or an unexpected one — as coffee in January (¥6,062.00) dominates our table. (2) Sum answers “how much in total?”; average answers “how much per sale?” — a group can have a large total from many small sales yet a low average, so the two rankings can differ. (3) A trailing-space value becomes a **separate** group, splitting one product’s revenue across two rows; you spot it because a familiar total is suddenly split or lower than expected.

7. Visualization

7.1. The right chart answers the question; the wrong one misleads

A chart turns numbers into a picture, and choosing the chart type **is** choosing which question you answer. Three types cover most early data work, each matched to a different shape of question.

Definition 13 (Three chart types, three questions).

- A **bar chart** compares a quantity **across categories** (revenue by product).
- A **line chart** shows how a quantity **changes along an ordered axis**, usually time (revenue by month).
- A **scatter plot** shows the **relationship between two numeric variables**, one on each axis (ad spend vs sales).

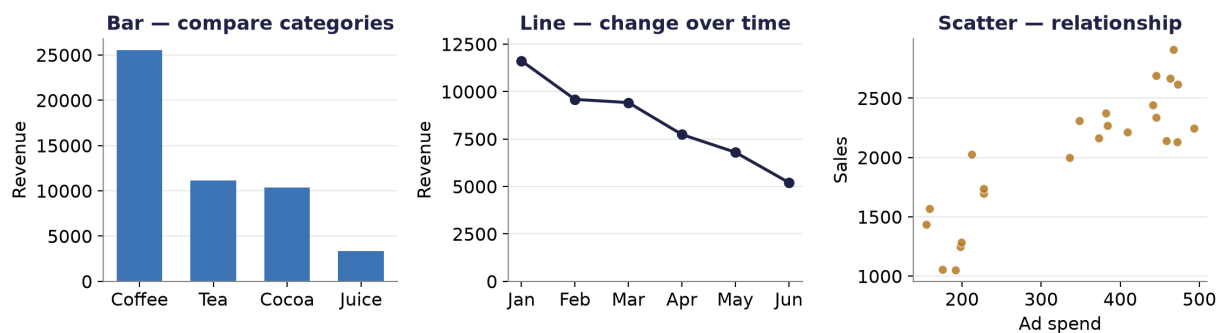


Figure 9: The café's real data in each chart type. **Bar:** coffee dwarfs the other products. **Line:** total revenue falls steadily from January to June, the seasonal decline of a hot-drink shop. **Scatter:** across 24 weeks, the weeks with higher ad spend tend to have higher sales — each dot is one week. Each chart fits a different question; using one where another belongs misstates the question.

Method 8 (Construct a chart).

1. Select the columns to plot. For a **bar** or **line** chart, select the **label** column — the category, or the time — **and** the **value** column. For a **scatter** plot, select the **two numeric columns** instead, one for each axis (Ad spend and Sales), with no label column at all (Figure 10). Hold Ctrl/Cmd to add a column that does not sit next to the first.
2. Open **Insert** → **Chart**. The spreadsheet proposes a type; change it to bar, line, or scatter to match your question.
3. Label the axes, and for a **bar** chart make sure the value axis starts at **zero** (see the pitfall below).

	A	B	C
1	Week	Ad spend	Sales
2	1	445	2689
3	2	373	2164
4	3	198	1249
5	4	472	2131

Insert → Chart
→ scatter plot:
ad spend on x, sales on y

Figure 10: Selecting the columns for a **scatter** plot: the two **numeric** columns (Ad spend and Sales), one per axis, and no label column. A bar or line chart would instead take one label column plus one value column.

Example — Matching chart to question.

- “Which product sells most?” — a comparison across categories → **bar chart**, and Figure 9 shows coffee far ahead.
- “Are sales rising or falling?” — change over an ordered time axis → **line chart**, and the café’s line falls month by month.
- “Does spending more on ads bring more sales?” — a relationship between two numbers → **scatter plot**.

Put months on a bar chart and you lose the sense of a trend; put unordered products on a line chart and the connecting line implies a progression that does not exist. The chart type is part of the claim, not decoration.

Pitfall. A misleading chart is as dangerous as a wrong calculation. The classic deception is a bar chart whose value axis starts not at zero but at, say, 95: a rise from 96 to 100 then **looks** like a huge jump. The data is honest; the axis misleads. When you read a chart, check the axes first; when you make a bar chart, start the value axis at zero unless you have a stated reason not to.

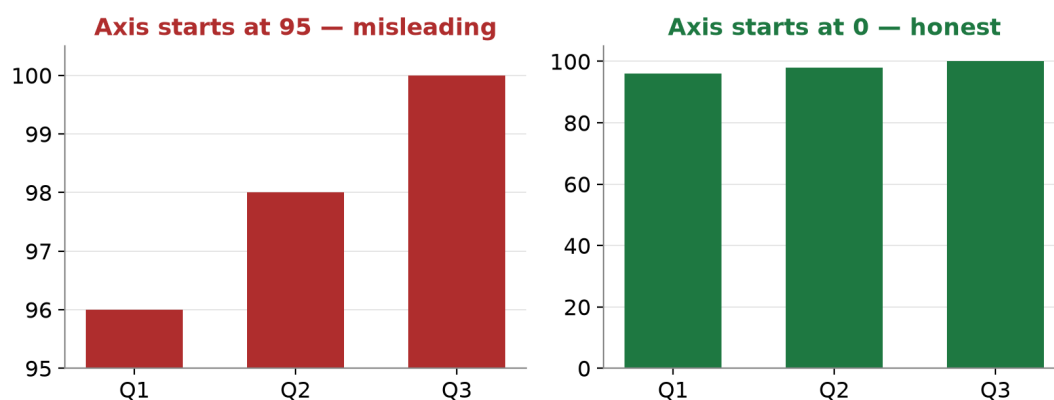


Figure 11: The same three values (96, 98, 100) drawn with the value axis starting at 95 (left) and at 0 (right). The numbers are identical; only the left chart exaggerates the change.

Exercises

7.1. From your data, construct a bar chart, a line chart, and a scatter plot. For each, state the question it answers and explain why the other two chart types would answer it less well.

7.2. Take any near-flat series and draw it twice: once with the value axis starting at zero, once starting just below the lowest value. Describe how the visual impression changes though the data does not.

7.3. Find or invent a chart that misleads, name exactly how (truncated axis, a line joining unordered categories, or similar), and redraw it honestly.

Answers. (1) A bar chart compares categories, a line chart shows change over ordered time, a scatter plot shows a relationship; each other type would blur or invent structure the question does not ask about. (2) The truncated axis stretches small differences into apparently large ones; the zero-based axis shows the change at its true, small size — same numbers, opposite impression. (3) The honest redraw restores a zero-based axis or replaces a line over categories with bars, so the picture matches the data.

8. From Business Question to Spreadsheet Answer

8.1. The whole course, in three steps

Everything so far — entry, functions, lookups, sorting, cleaning, pivots, charts — serves one end: answering a real question that someone actually has. Doing that well is a disciplined three-step move.

Definition 14 (Formulate, select, interpret).

1. **Formulate** the question precisely enough that you would recognise the answer if you saw it (“which product should we promote for the summer?”, not “how are products doing?”).
2. **Select** the operations that answer exactly that — the functions, sorting, filtering, pivot, or chart the question needs, and no more.
3. **Interpret** the result back in the question’s own words, stating what it means rather than reporting a bare number.

Case study — Which product to promote for the summer. Formulate. The café’s revenue falls from winter to summer. We want the product whose sales **hold up** or **grow** into summer — the one worth promoting when the others fade.

Select. Build a pivot of revenue by product and month (Figure 8). Read across each product’s row from January to June. Coffee falls from ¥6,062.00 toward summer with the other hot drinks; juice moves the other way, from ¥304.50 in January to ¥819.00 in June. A line chart of the two rows makes the crossing obvious.

Interpret. “Coffee earns the most overall, but its sales decline through spring; juice is the only product that **grows** into summer. Promote juice for the summer months, when it is rising and the hot drinks are not.” Notice what we did **not** do: no scatter plot, no average across all products, no extra cleverness. The discipline is to use only the operations the question needs, and to end with a sentence, not a number.

Pitfall. The commonest failure is producing numbers without an answer — a wall of sums and charts that never returns to the question that was asked. A correct total that does not answer the question is not a partial answer; it is no answer. Always close the loop back to the words of the question.

Exercises

8.1. Formulate one genuine business question about your dataset, precise enough that you would recognise the answer. Then list the exact operations you will use to answer it, and nothing more.

8.2. Answer your question using only those operations, and finish with a single interpretive sentence phrased in the question's own terms.

8.3. Take a vague question ("how are we doing?") and rewrite it as a precise, answerable one.

Explain what made the first version impossible to answer.

Answers. (1) A good question names the comparison, the measure, and the scope (product, revenue, summer months), so the answer is recognisable; the operation list should contain only what that question needs. (2) The interpretive sentence restates the finding in business terms, as "promote juice for the summer" does for the case study. (3) "How are we doing?" names no measure, scope, or comparison, so no operation can answer it; a precise version supplies all three.

9. Documentation and Reproducibility

9.1. An analysis no one can repeat is not finished

The last discipline separates a private spreadsheet from professional work: someone else — or you, six months later — must be able to follow it and reach the same result **without asking you anything**.

Definition 15 (Reproducible analysis). A spreadsheet analysis is **reproducible** when another person can follow it, understand each step, and reach the same result without assistance. This means recording the **source** of the data, the **steps** taken in order, the **assumptions and judgements** made (especially how data-quality issues were handled), and **where each reported number comes from**.

Example — The difference a note makes. Two analysts both report an average sale of ≈ 69.57 . The first hands over a sheet of formulas. The second adds a short **notes** tab: “Source: café sales export, six months, 724 rows. Average computed over E2:E725. No rows excluded.” Only the second analysis can be **trusted**, because only the second can be **checked**. The number is identical; the work is not.

Method 9 (Write a notes tab that makes an analysis reproducible). On a separate sheet named Notes, record, in order:

1. **Source** — where the data came from and when it was obtained.
2. **Cleaning** — every correction made and the judgement behind it (duplicates removed, spellings standardised, blanks excluded or filled).
3. **Steps** — the operations performed, in the order performed.
4. **Results** — each reported number and the exact cell range it comes from.

Example — A notes tab for the cleaned extract. Documenting the cleaning of Chapter 5’s dirty extract: “Source: February extract, 12 rows. Removed 1 exact duplicate row so 11 remain. Standardised the Product column: corrected the typo Cofee, trimmed the trailing space in “Cof fee”, and lower-cased coffee to Coffee for consistency (4 Coffee rows in total; case never affected the count). One Units cell is blank — genuinely unknown, so excluded from unit totals.” Anyone with the raw extract and these notes reaches the same cleaned numbers we did.

Pitfall. Undocumented cleaning is the quiet destroyer of reproducibility. If you removed duplicates, standardised spellings, or excluded blanks (Chapter 5) without writing it down, your final numbers cannot be rebuilt from the raw data — and an analysis that cannot be rebuilt cannot be defended when someone questions it. Document the judgements, not just the formulas.

Exercises

9.1. Write a notes tab for one of your earlier analyses: source, cleaning, steps, and the cell range behind each reported number.

9.2. Hand your raw data and your notes tab — but **not** your finished sheet — to a classmate, and confirm they reproduce your headline number without asking you anything. Note any step they got stuck on and add it to your notes.

9.3. Explain in a few sentences why two analysts reporting the **same** number can differ in how much their result can be trusted.

Answers. (1) A complete notes tab lets a stranger rebuild every number; the test is whether each reported figure names its source range and every cleaning judgement. (2) A step the classmate stumbles on is a gap in the notes — the exercise's real output is the improved notes. (3) The same number can rest on documented, checkable steps or on undocumented ones; only the first can be verified, so only the first can be trusted, as in the two-analyst example.